

TKinter

INTRODUCTION

- ❑ Tkinter is Python's standard GUI (Graphical User Interface) library.
- ❑ It is a built-in module, so you don't need to install it separately.
- ❑ provides a simple way to create desktop applications with windows, buttons, labels, text boxes, menus, and more — all using pure Python code.
- ❑ The name "Tkinter" comes **from Tk (the original GUI toolkit) + inter (interface)**, meaning Python's interface to the Tk GUI library.



KEY FEATURES OF TWINTER

- ❑ Completely free and open-source
- ❑ Cross-platform (works on Windows, macOS, Linux)
- ❑ Lightweight and fast
- ❑ Easy to learn for beginners
- ❑ Supports buttons, labels, entry fields, checkboxes, menus, canvas, dialogs, etc.

Why Tkinter?

Pros

- ❑ Completely free and open-source
- ❑ Cross-platform (works on Windows, macOS, Linux)
- ❑ Lightweight and fast
- ❑ Easy to learn for beginners
- ❑ Supports buttons, labels, entry fields, checkboxes, menus, canvas, dialogs, etc.

Cons

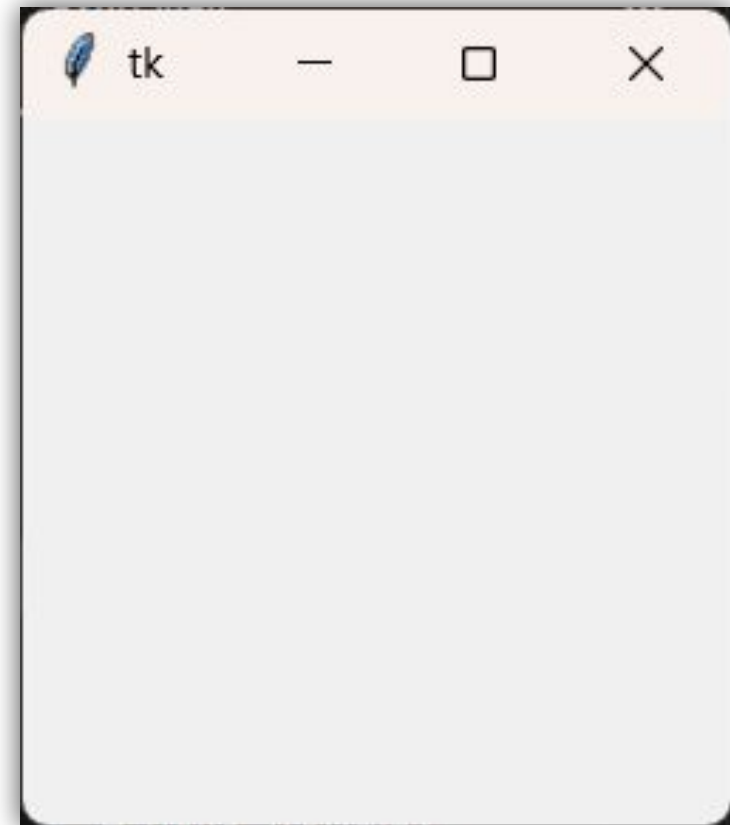
- ❑ Outdated, "old-school" appearance
- ❑ Limited for Complex / Modern UI
- ❑ Not Designed for Heavy Applications
- ❑ Single-threaded GUI Loop
- ❑ Fewer Built-in Components than Others

CREATING AN EMPTY Tkinter WINDOW

An empty Tkinter top-level window can be created by using the following steps.

- ❑ import the Tkinter module.
- ❑ Create the main application window.
- ❑ Add the widgets like labels, buttons, frames, etc. to the window.
- ❑ Call the main event loop so that the actions can take place on the user's computer screen.

```
from tkinter import *  
root = Tk()  
root.mainloop()
```



WINDOW METHODS

Method	Purpose / Definition	Example
title()	Sets the window title (text on title bar)	<code>root.title("My App")</code>
geometry()	Sets window size & position	<code>root.geometry("600x400+200+100")</code>
resizable()	Enables/disables window resizing	<code>root.resizable(False, True)</code>
minsize()	Sets minimum window size	<code>root.minsize(300, 200)</code>
maxsize()	Sets maximum window size	<code>root.maxsize(800, 600)</code>
iconphoto()	Sets window icon	<code>root.iconphoto(True, img)</code>
config() / configure()	Changes window properties (bg, cursor, etc.)	<code>root.config(bg="lightblue")</code>
state()	Controls window state (normal/minimized/maximized)	<code>root.state("zoomed")</code>
attributes()	Sets special attributes (fullscreen, transparency, topmost)	<code>root.attributes("-alpha", 0.8)</code>
withdraw()	Hides the window	<code>root.withdraw()</code>
deiconify()	Shows hidden/minimized window	<code>root.deiconify()</code>
lift()	Brings window to the front	<code>root.lift()</code>
lower()	Sends window behind others	<code>root.lower()</code>
focus_force()	Forces window to take focus	<code>root.focus_force()</code>
destroy()	Closes the window completely	<code>root.destroy()</code>

To control the behavior, size, appearance, and properties of the main application window (Tk()) or secondary windows (Toplevel()).

- They allow you to:
- Set window size
- Change title
- Enable/disable resizing
- Set icons
- Control visibility
- Change appearance
- Close the window

Tkinter WIDGETS

- ❑ Tkinter widgets are the building blocks of a GUI application.
- ❑ Widgets are the visual elements (components) that appear on the screen.
- ❑ Example: buttons, labels, text boxes, frames, menus, etc.

Widget	Definition / Purpose
Label	Displays text or images
Button	Creates a clickable button
Entry	Single-line text input
Text	Multi-line text input
Frame	Container to hold and group other widgets
LabelFrame	Frame with a label border
Checkbutton	Checkbox (on/off option)
Radiobutton	Radio button (select one option)
Listbox	Displays list of selectable items
Scrollbar	Adds scrolling to widgets
Canvas	For drawing shapes, images, graphics
Spinbox	Input with up/down arrows
Scale	Slider widget for numerical values
Menubutton	Dropdown menu button
Menu	Creates menus and submenus
Message	Auto-wrapping text display (like Label)
Toplevel	Creates a new window
PanedWindow	Adjustable segmented window
OptionMenu	Dropdown selection
Combobox (ttk)	Advanced dropdown with typing support
Treeview (ttk)	Table/list/tree structure
Notebook (ttk)	Tabbed interface
Progressbar (ttk)	Loading/progress indicator

Common Options

Option	Meaning
text	Text to display
font	Font style/size
Fg	Text color
bg	Background color
image	Display an image
compound	Combine text + image (e.g., LEFT, TOP)
padx, pady	Padding inside the label
width, height	Size of label

1. Label()

Used to display **text**, **images**, or **both** inside a window or frame.

Purpose

A Label is used for:

- ❑ Displaying static text
- ❑ Showing instructions or titles
- ❑ Displaying images (PNG, JPG, GIF)
- ❑ Displaying dynamic text (updated using `.config()`)

Syntax

- ❑ `Label(master, text="Your Text", options...)`

Common Options

Option	Meaning
width	Number of characters wide
font	Text font
fg	Text color
bg	Background color
show	Hides characters (useful for passwords), e.g., show="*"
state	normal, disabled, or readonly
textvariable	Links Entry to a StringVar

2. Entry()

Used to create a **single-line text input field**.

it allows the user to type or edit text, such as:

- ❑ Username
- ❑ Password
- ❑ Search text
- ❑ Numbers
- ❑ Small forms
- ❑ It is the standard widget for text entry in Tkinter.

Syntax : **Entry(master, options...)**

3. Button()

- ❑ Used to create a **clickable button**
- ❑ It performs an **action or function when the user clicks it.**

A Button is used to trigger:

- ❑ Functions or commands
- ❑ Opening windows
- ❑ Submitting forms
- ❑ Closing the app
- ❑ Starting processes
- ❑ Updating widgets

Syntax:

Button(master, text="Label", command=function, options...)

Parameter	Meaning
master	Parent widget (root, Frame, Toplevel, etc.)
text	Text to display on the button
command	Function to execute when button is clicked
options	Styling/customization options

Common Options

text , font , fg , bg , width , height , command , image , state

Common Options

Option	Meaning
width	Number of characters shown horizontally
height	Number of lines shown vertically
font	Font style and size
bg	Background color
fg	Text (foreground) color
padx, pady	Internal padding
bd	Border width
relief	Border style (flat, groove, ridge, solid, etc.)
yscrollcommand	Connect vertical scrollbar
xscrollcommand	Connect horizontal scrollbar
wrap	Line wrap mode (none, char, word)
state	normal (editable) or disabled (read-only)

4. Text()

Used to create a multi-line text input area where the user can type, display, and edit large amounts of text.

Characteristics

- ❑ Multi-line input
- ❑ Support very long text
- ❑ Text formatting (colors, fonts)
- ❑ Scrollbars
- ❑ Copy / Paste / Undo
- ❑ Images & hyperlinks

Syntax : **Text(master, options.....)**

5. Radio Button

- ❑ Used for **mutually exclusive selection**—meaning **only one choice at a time**.
- ❑ Allows the user to **select only one option from a group of choices**.
- ❑ **Key Point** : All radio buttons in a group must be linked to the **same Tkinter variable** (such as IntVar, StringVar, BooleanVar, etc.).
- ❑ **Syntax**
Radiobutton(parent, text="label", variable=var, value=option_value, **options)

Option	Purpose
font	Custom font
fg / bg	Text & background color
padx / pady	Internal padding
state=DISABLED	Disable button
indicatoron=0	Convert radio button into a toggle-like button

Variable type in Tkinter

- ❑ Tkinter widgets **cannot store Python variables directly**.
- ❑ Tkinter provides special variable wrappers that:
 - ✓ Store data selected or entered in widgets
 - ✓ Automatically update when the user interacts
 - ✓ Let you retrieve values easily using `.get()`
 - ✓ Allow you to assign values using `.set()`

Variable Type	Data Stored	Example Usage
IntVar()	Integer values	Radio buttons with numeric options, counters
StringVar()	String values	Entry fields, radio buttons with text
BooleanVar()	True/False values	Checkbuttons / switches
DoubleVar()	Floating point numbers	Sliders / numeric inputs
Variable()	Generic type	Rarely used; auto type

Syntax: `frame =Frame(parent,**options)`

Common Options

Common Options	Description
bg	Background color
width / height	Size of frame
bd	Border size
relief	Border style (flat, raised, sunken, groove, ridge)
padx / pady	Internal padding

6. Frame

- ❑ A **container widget** used to group and organize other widgets inside a window.
- ❑ Think of it like a **section** or **box** inside the main window that holds related widgets together.
- ❑ It does not display text or images — its only purpose is **layout and organization**.
- ❑ Frames help to:
 - ◆ Organize widgets into logical groups
 - ◆ Manage complex layouts cleanly
 - ◆ Create sidebars, headers, footers, content areas
 - ◆ Use different layout managers inside different areas (pack, grid, etc.)

Common Options

Option	Meaning
text	Title of the frame
font	Font of title text
fg	Title color
bg	Background color
labelanchor	Title position (n, ne, nw, e, w, etc.)
padx / pady	Inside padding
relief	Border style (groove, ridge, etc.)

7. LabelFrame

- ❑ A container widget (like Frame) but with a border and a title (label) around it.
- ❑ It is used to group related widgets together with a **caption heading**, making the UI more structured and organized.
- ❑ **Syntax : `labelframe = LabelFrame(parent, text="Title Here", **options)`**
- ❑ It helps to:
 - ◆ Organize widgets visually
 - ◆ Group related input sections
 - ◆ Add a title / category for clarity
 - ◆ Improve professional UI look

8. Check Button

- ❑ A **CheckButton** is a toggle-style button that represents a **binary choice** — ON/OFF, TRUE/FALSE, YES/NO.
- ❑ Allows the user to **select or deselect an option**.
- ❑ Unlike RadioButtons, **multiple CheckButtons can be selected at the same time**.
- ❑ **Syntax**
Checkbutton(parent, text="label", variable=var, onvalue=value_when_checked, offvalue=value_when_unchecked, command=function)

Option	Use
text	Text displayed beside the check button
variable	Links the checkbutton to a Tkinter variable (IntVar, BooleanVar, StringVar)
onvalue	Value stored when the checkbutton is checked
offvalue	Value stored when the checkbutton is unchecked
command	Function executed when the user checks/unchecks
state	Enable/disable widget (NORMAL, DISABLED)
bg	Background color
fg	Text color
font	Font of the label text
padx / pady	Widget padding

9. Listbox

❑ Used to **display a list of items** from which the user can **select one or multiple options**.

❑ **Key Features**

❑ Shows multiple values/items inside one widget

❑ Supports **single** and **multiple** selections

❑ Can be used with a **Scrollbar**

❑ Items can be **added, removed, updated** dynamically

❑ **Syntax : `listbox = Listbox(parent, **options)`**

Option	Meaning
selectmode	Selection type (SINGLE, BROWSE, MULTIPLE, EXTENDED)
height	Number of rows displayed
width	Width of Listbox
bg / fg	Background & text colors
font	Font of items
activestyle	Style of selected item highlight

Common Options

Option	Description
orient	Defines scroll direction: VERTICAL (default) or HORIZONTAL
command	Links the scrollbar to the widget being scrolled
bg / background	Background color
activebackground	Color when mouse is over scrollbar
troughcolor	Color of the scrollbar track
width	Width of scrollbar

10. Scrollbar

- ▣ Allows the user to **scroll** another widget when its content is too large to fit inside the visible area.
- ▣ **Syntax : `scroll = Scrollbar(parent, options)`**

11. Canvas

- ▣ Used to **draw and display graphics**.
- ▣ It allows you to create **shapes, images, charts, custom UI elements, animations, and even games**.
- ▣ **Syntax** : `canvas = Canvas(parent, options...)`

Use Case	Example
Drawing shapes	Lines, circles, rectangles
Displaying images	Photo viewer
Animations	Moving objects (e.g., bouncing ball)
Custom UI	Custom buttons, dashboards
Games	Tic-tac-toe, car racing
Scrollable area	scrollable frame inside canvas

Common Options

Option	Description
from_	Starting value of range
to	Ending value of range
increment	Step value when clicking arrows (default = 1)
values	Instead of numeric range, provide a tuple/list of custom values
width	Width of the box
textvariable	Variable to store the selected value (IntVar / StringVar)
command	Function to call when value changes
state	normal, disabled, or readonly
font	Font style

12. Spinbox

- ❑ Allows the user to select a value by **clicking up/down arrow buttons** or by **typing manually**.
- ❑ It is mainly used when the valid input is within a **fixed numeric range** or a **list of predefined values**.
- ❑ **Syntax : `Spinbox(master, from_=start, to=end, **options)`**

13. Menubutton

- Displays a button which, when clicked, shows a **dropdown menu**.
- It is used to provide a compact list of options, similar to a **menu bar item** or a **dropdown selector**.
- **Syntax : Menubutton(master, **options)**

Option	Description
text	The label displayed on the button
menu	Links the Menubutton to a Menu widget
bg / fg	Background and text color
font	Font style
direction	Direction of dropdown (above, below, left, right, flush)
relief	Button border style (flat, raised, sunken, etc.)
width	Width of the menubutton
state	normal, active, or disabled

Common Options

Option	Description
tearoff	If 0, removes the dashed line at the top (commonly used)
bg / fg	Background and text color
font	Font style
activebackground	Background on hover
activeforeground	Text color on hover
disabledforeground	Text color when disabled

14. Menu

- ❑ Used to create **drop-down menus** such as **Menu Bar**, **Right-click (Context) Menu**, or **Dropdown for Menubutton**.
- ❑ It holds one or more **menu items** like commands, separators, checkbuttons, and radiobuttons.
- ❑ **Syntax** : Menu(master, **options)

15. Message

- ❑ Used to display **multiple lines of text** that **automatically wrap** to fit the given width.
- ❑ It is similar to a **Label**, but **better suited for long messages**, paragraphs, descriptions, or notifications.
- ❑ The **Message widget is less commonly used today**, because many developers use **Label with wraplength option** instead.
- ❑ **Syntax** : `Message(master, text="", **options)`

Option	Description
text	Text to display
textvariable	Variable (StringVar) to update message dynamically
width	Width of the message in pixels (controls wrapping)
bg / fg	Background and text color
font	Font style
relief	Border style (flat, ridge, sunken, etc.)
padx / pady	Internal padding
anchor	Text alignment inside (n, s, e, w, center)

Common methods / options

Method / Option	Purpose
<code>title("...")</code>	Set window title
<code>geometry("300x200")</code>	Set window size
<code>resizable(False, False)</code>	Disable resizing
<code>iconbitmap("icon.ico")</code>	Set window icon
<code>attributes('-topmost', True)</code>	Keep window always on top
<code>protocol("WM_DELETE_WINDOW", func)</code>	Custom close function

16. Toplevel

- ❑ Used to create a **new independent window** separate from the main application window.
- ❑ It behaves like a **popup window / child window**, but can contain any widgets like buttons, labels, frames, etc.
- ❑ **Syntax** : `Toplevel(master)`
- ❑ Features
 - ❑ Opens a new desperate window
 - ❑ Same event loop as root
 - ❑ Can have title, size, icons
 - ❑ Can be shown or hidden

17. PanedWindow

- ❑ A **container widget** that holds one or more **resizable panes**.
- ❑ Each pane can contain another widget, and the panes can be resized by dragging a **movable splitter (handle)** between them.
- ❑ **Syntax** : `PanedWindow(master, **options)`

Option	Description
orient	Direction of panes → HORIZONTAL or VERTICAL
showhandle	Shows handle for resizing (Boolean)
sashrelief	Border style of divider/sash
sashwidth	Width of the divider bar
bg	Background color
width, height	Initial size of the paned window

Key Methods

Method	Purpose
add(widget)	Adds a new pane
remove(widget)	Removes a pane
panes()	Returns list of current panes
forget(widget)	Removes widget but does not destroy it

Common methods / options

Option	Description
bg / fg	Background and text color
font	Font style
width	Width of the OptionMenu
highlightthickness	Border highlight
state	normal or disabled

18. OptionMenu

- ❑ Provides a **dropdown list** of predefined choices from which the user can select **one value at a time**.
- ❑ It is a simple and user-friendly alternative to **Combobox** or **Menubutton with Menu**.
- ❑ **Syntax** : `OptionMenu(master, variable, *values)`

19. Combobox

- ❑ **Combobox in Tkinter (from ttk module)** is a widget that provides a **drop-down list** of options **with the ability to type or select**.
- ❑ It is more **modern and flexible** than OptionMenu, and supports **autocomplete-style typing**.
- ❑ **Syntax**

```
from tkinter.ttk import Combobox  
Combobox(master, **options)
```

Option	Description
values	List/tuple of items shown in dropdown
textvariable	Variable that stores selected/typed value
state	"normal" (editable), "readonly" (selection only), "disabled"
width	Width of the entry area
font	Font style (inherited from ttk theme unless changed)

Common modes

Mode	Usage
tree	Hierarchical structure (example: folders)
headings	Table mode (only columns)
tree headings	Both hierarchy + columns

Method	Purpose
insert(parent, index, text, values)	Add a new row
item(item_id)	Get or update row data
delete(item_id)	Delete row
get_children(item_id)	Get child rows
selection()	Return selected row(s)
move(item, parent, index)	Move item to another position
focus()	Returns the currently focused item

20. Treeview

- ❑ A **Treeview in Tkinter (from ttk module)** is a powerful widget used to display **data in a tabular or hierarchical (tree-like) structure**.
- ❑ It is commonly used for **tables, file explorers, category hierarchies, and database records**.
- ❑ **Import Syntax:** `from tkinter.ttk import Treeview`
- ❑ **Features**
 - ❑ Hierarchical data
 - ❑ Tabular data
 - ❑ Editable selection
 - ❑ Scrollbar supported
 - ❑ Custom width & heading

21. Progressbar

- ❑ A **Progressbar** is a widget in Tkinter (from the ttk module) that **visually represents the progress of a long-running task**.
- ❑ It shows a bar that fills up over time.
- ❑ Progressbars are especially useful when performing tasks like downloading, uploading, loading files, database operations, etc.
- ❑ `from tkinter.ttk import Progressbar`

Important Options

Option	Description
orient	Orientation of the bar: "horizontal" (default) or "vertical"
length	Width/height of bar in pixels
mode	"determinate" or "indeterminate"
maximum	Maximum value (default: 100)
value	Current progress value

21. Progressbar

Modes of Progressbar

Mode	Description
determinate	Used when the progress amount is known. The bar fills to a defined maximum.
indeterminate	Used when task duration is unknown. The bar moves back and forth continuously.

Important Methods

Method	Description
<code>.start(interval)</code>	Starts indeterminate progress animation
<code>.stop()</code>	Stops indeterminate animation
<code>.step(amount)</code>	Increases progress by given value
<code>.config()</code>	Update properties dynamically